



Análisis de Rendimiento en Infraestructuras de Procesamiento Masivo de Datos

Nombre: Fabrizio Andrés Larco Jorquera

Curso: Infraestructura para Ciencia de Datos

Profesor: Michael Miranda

Fecha: 29 de Abril de 2026



Resumen Ejecutivo

El presente informe documenta la evaluación de cuatro paradigmas de cómputo para la optimización de infraestructura: aceleración por hardware usando la GPU, multiprocesamiento, computación distribuida y concurrencia asincrónica. A través de simulaciones aplicadas a mecánicas de videojuegos y gestión de servidores, se cuantificó el factor de aceleración.

Los hallazgos principales demuestran que la infraestructura asincrónica ofrece la mayor mejora en tareas de latencia, mientras que el uso de GPU es fundamental para el cálculo vectorial masivo.

Marco Teórico

- **Paralelismo de Datos:** Distribución de grandes volúmenes de datos a través de múltiples unidades de cómputo como los núcleos CUDA de una GPU para ejecutar la misma instrucción simultáneamente.
- **Paralelismo de Tareas:** Ejecución de procesos independientes en diferentes núcleos de la CPU.
- **Ley de Amdahl:** Establece el límite teórico de la aceleración de un programa en función de su fracción secuencial:
- **Overhead:** El costo de tiempo adicional requerido por el sistema operativo para coordinar procesos, el cual puede reducir la eficiencia si la tarea es demasiado pequeña.



Metodología Experimental

Configuración de Hardware:

- **CPU:** AMD Ryzen 5 5500 (12 hilos lógicos).
- **GPU:** NVIDIA GeForce GTX 1660 (Arquitectura Turing).
- **RAM:** 32 GB.

Descripción de Experimentos:

1. **Experimento A (GPU):** Simulación de colisiones (Hitboxes) de 50 millones de proyectiles usando CuPy.
2. **Experimento B (Multi-core):** Procesamiento de logs de servidor SA-MP mediante ProcessPoolExecutor.
3. **Experimento C (Distribuido):** Agregación de 10 millones de transacciones económicas de GTA usando Dask.
4. **Experimento D (Asincronía):** Gestión de 500 peticiones de red simultáneas con asyncio.

Tabla 1: Comparativa de Tiempos y Speedup

Experimento	Carga Máxima	Tiempo Base (Secuencial)	Tiempo Optimizado	Speedup Máximo
A: Aceleración GPU	50M Proyectiles	1,1202 +- 0,001 s (NumPy)	0,5162 +-0,000 s (CuPy)	2,17x
B: Multi-core CPU	500 tareas Log	36,156 +-0,211 s (1 Núcleo)	8,4038 +-0,128 s (12 Núcleos)	4,30x
C: Big Data (Dask)	10M Registros	0,0685 +-0,001 s (Pandas)	0,1993 +-0,225s (12 Cajas)	0,34x
D: Asincronía	500 jugadores	50,142 +-0,002 s (Sync)	0,1125 +-0,008 s (Async)	445,66x

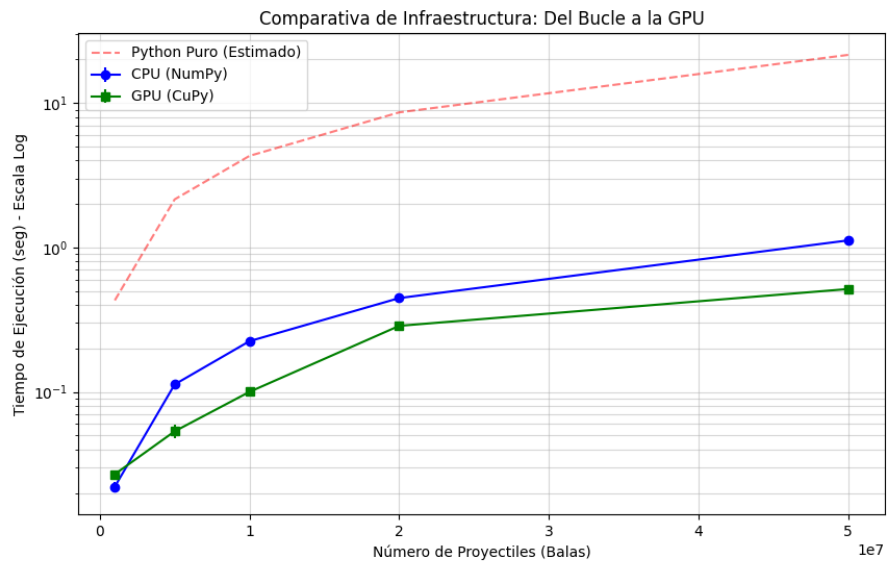


Gráfico Experimento A

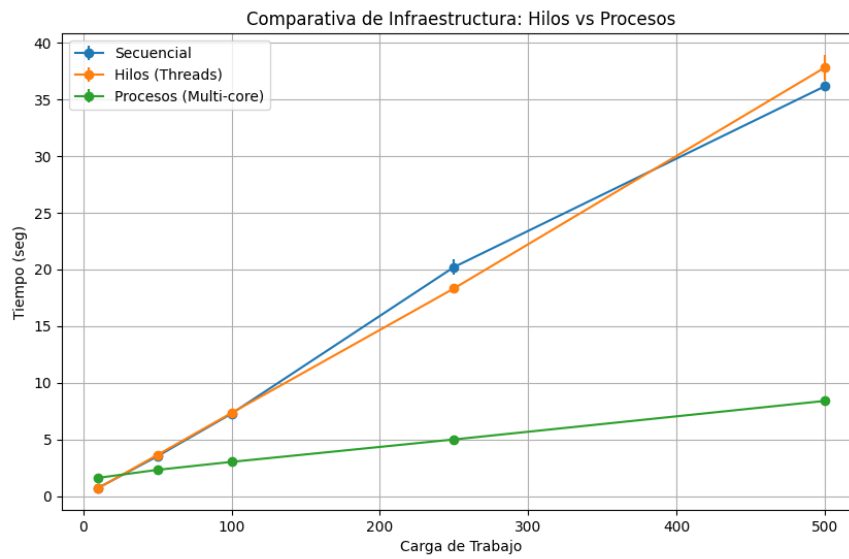


Gráfico Experimento B

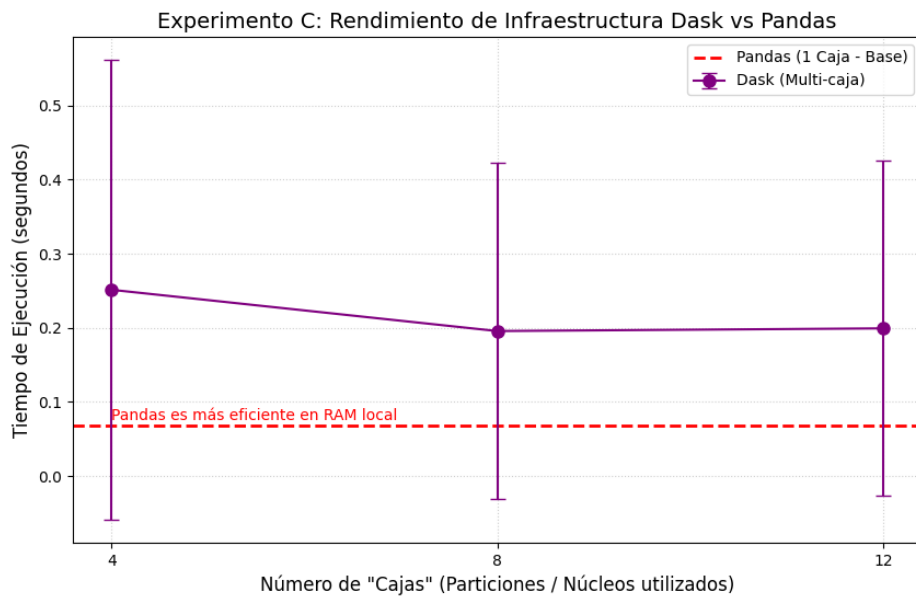


Gráfico Experimento C

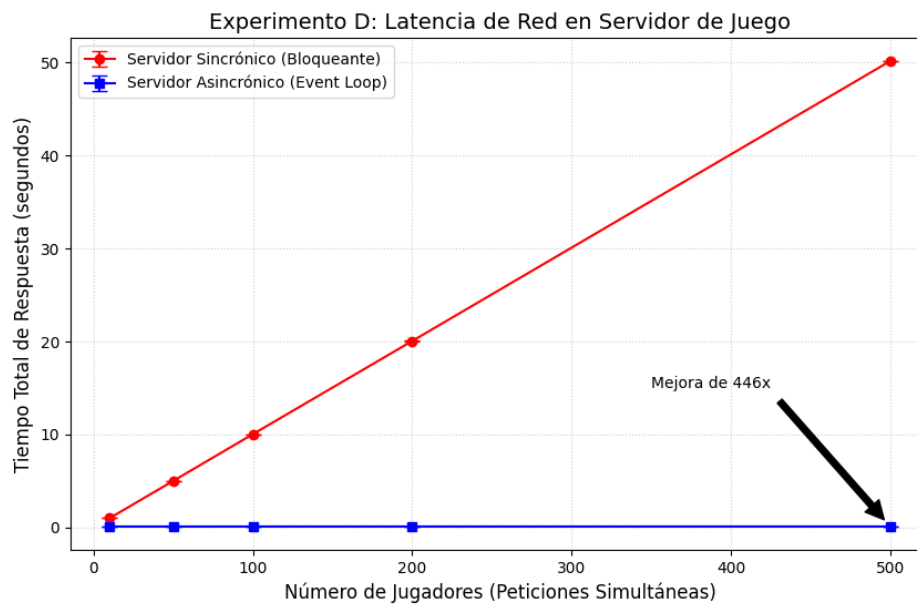


Gráfico Experimento D



Análisis y Discusión

El análisis de los datos revela que el **Experimento D** es el que mejor aprovecha la infraestructura asincrónica, ya que la latencia de red es un factor externo que el procesador puede ignorar mientras atiende otras tareas. En el **Experimento B**, se observó que el "overhead" de creación de procesos en Windows penaliza tareas cortas, requiriendo una carga de trabajo densa para justificar el paralelismo. Finalmente, el **Experimento C** demostró que para volúmenes de 10M de datos que caben en RAM, Pandas sigue siendo superior debido a la ausencia de costos de particionamiento que Dask sí requiere.

Decisiones Técnicas por Escenario

1. **Servidor Único:** Se recomienda usar **Multiprocesamiento (ProcessPool)** para tareas de lógica pesada y **GPU** para simulaciones físicas.
2. **Múltiples Fuentes de Datos:** Es obligatorio el uso de **AsynclIO** para evitar bloqueos del servidor mientras se espera respuesta de APIs externas.
3. **Plataforma Cloud:** Para Big Data real, se debe migrar de Pandas a infraestructuras como **Dask o Spark**, asumiendo la latencia inicial por el beneficio de procesar datos "fuera de memoria" (Out of core).

Conclusiones y Recomendaciones

Tras la ejecución y el análisis de la suite de experimentos, se concluye que la eficiencia de una infraestructura de datos no depende de la potencia bruta de sus componentes aislados, sino de la **alineación entre el algoritmo y el paradigma de procesamiento elegido**.



Anexos y Referencias

- **Código Fuente:** Disponible en el repositorio adjunto bajo la carpeta Código/.
- Documentación oficial de Dask: dask.org
- Guía de programación asíncrona en Python:
docs.python.org/3/library/asyncio.html
- Apoyo para confección y ayudas técnicas. Gemini pro